

04주차 · 문맥 임베딩과 Transformer

- 연계 슬라이드: [04주차 슬라이드](#)
- 실습 Notebook: [week04.ipynb](#)
- 교재 연계: *Hands-On Large Language Models* 2장 일부와 3장
- 권장 학습 순서: 강의노트 읽기 → 어텐션 손계산 → Notebook 기본 셀 → 선택 BERT 실습

1. 이번 주에 답할 질문

“배를 먹었다”의 배와 “배를 탔다”의 배는 글자와 토큰 ID가 같을 수 있다.

그러나 첫 문장에서는 과일을, 둘째 문장에서는 선박을 뜻한다. 하나의 단어에 하나의 벡터만 부여하는 정적 임베딩으로는 이 차이를 입력 단계에서 직접 표현하기 어렵다.

문맥 임베딩(contextual embedding)은 한 토큰의 표현을 그 토큰만으로 정하지 않고, 문장 안의 다른 토큰을 함께 고려해 계산한다. 따라서 위치 i 의 표현은 다음처럼 문장 전체의 함수로 쓸 수 있다.

$$\mathbf{h}_i = f_{\theta}(w_1, w_2, \dots, w_T)_i \quad (1)$$

여기서 $w_{1:T}$ 는 길이 T 의 토큰 열이고, $\mathbf{h}_i$ 는 위치 i 의 은닉 상태(hidden state)다. 같은 표면형이라도 주변 문맥이 달라지면 $\mathbf{h}_i$ 도 달라진다.

이번 주의 핵심 질문은 다음과 같다.

1. 입력 토큰 임베딩은 어떻게 문맥 은닉 상태로 바뀌는가?
2. 자기 어텐션(self-attention)은 어떤 정보를, 어떤 가중치로 섞는가?
3. Transformer 블록에서 위치 정보·잔차 연결·층 정규화·피드포워드 네트워크는 왜 필요한가?
4. 생성 모델의 은닉 상태와 검색용 문장 임베딩은 왜 같은 개념이 아닌가?

2. 학습목표

학습을 마치면 다음을 할 수 있어야 한다.

- 토큰 임베딩, 위치 표현, 문맥 은닉 상태를 구분한다.
- 질의(query), 키(key), 값(value)의 역할과 행렬 모양을 설명한다.
- 스케일 내적 어텐션의 점수, 소프트맥스, 가중합을 작은 행렬로 계산한다.
- 패딩 마스크와 인과 마스크가 서로 다른 문제를 해결함을 설명한다.
- 다중 헤드 어텐션과 Transformer 블록의 정보 흐름을 설명한다.
- BERT 계열과 GPT 계열의 마스크·학습목표·활용 차이를 비교한다.
- KV 캐시와 검색용 벡터 색인을 구분한다.
- 어텐션 가중치 그림을 인과적 설명으로 과해석하지 않는다.

3. 입력 임베딩은 아직 문맥을 모른다

Transformer의 입력 위치 i 는 보통 여러 표현의 합으로 시작한다.

$$\mathbf{x}_i = \mathbf{e}_{\text{token}(i)} + \mathbf{p}_i + \mathbf{s}_i \quad (2)$$

- $\mathbf{e}_{\text{token}(i)}$: 토큰 ID로 조회한 학습 벡터
- $\mathbf{p}_i$: 절대 또는 상대 위치를 나타내는 위치 표현
- $\mathbf{s}_i$: 문장 A/B 같은 입력 구획을 나타내는 표현이며, 모델에 따라 생략됨

토큰 임베딩 표에서 조회한 $\mathbf{e}_{\text{token}(i)}$ 는 같은 ID에 항상 같은 벡터를 준다.

“배를 먹었다”와 “배를 탔다”의 차이는 여러 Transformer 층을 통과한 뒤의 $\mathbf{h}_i$ 에

나타난다. 그러므로 `input_embeddings`와 `last_hidden_state`를 같은 것으로 부르면 안 된다.

입력 행렬을 $X \in \mathbb{R}^{T \times d}$ 라고 하자. 한 행은 한 토큰 위치, 한 열은 표현의

한 차원이다. 배치 차원을 포함한 실제 라이브러리 출력은 흔히 `[batch, tokens, hidden]` 모양을 가진다.

4. 자기 어텐션의 출발점: Q, K, V

자기 어텐션은 같은 입력 X 를 서로 다른 세 선형 변환에 통과시킨다.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (3)$$

$W_Q, W_K \in \mathbb{R}^{d \times d_k}$ 이고 $W_V \in \mathbb{R}^{d \times d_v}$ 라면,
 $Q, K \in \mathbb{R}^{T \times d_k}, V \in \mathbb{R}^{T \times d_v}$ 이다.

세 이름은 정보 검색에 비유하면 이해하기 쉽다.

- 질의 Q : 현재 위치가 어떤 단서를 찾는가?
- 키 K : 각 위치가 어떤 단서를 제공한다고 표시하는가?
- 값 V : 그 위치가 선택되었을 때 실제로 전달할 정보는 무엇인가?

질의와 키가 “같은 토큰에서 만들어졌다”는 사실은 두 벡터가 같다는 뜻이 아니다.
서로 다른 가중치 W_Q 와 W_K 를 학습하므로, 매칭에 필요한 관점도 달라질 수 있다.

5. 스케일 내적 어텐션을 단계별로 유도하기

스케일 내적 어텐션(scaled dot-product attention)은 다음과 같다.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (4)$$

5.1 호환성 점수

$QK^\top \in \mathbb{R}^{T \times T}$ 의 원소 (i, j) 는 질의 위치 i 와 키 위치 j 의
내적이다. 행 i 전체는 “위치 i 가 모든 위치를 얼마나 참고할 것인가”를 나타내는 원점수다.

5.2 왜 $\sqrt{d_k}$ 로 나누는가

질의와 키의 각 성분이 평균 0, 분산 1이며 대략 독립이라고 가정하면, 내적은 d_k 개의 곱을
더하므로 분산이 d_k 에 비례한다. $\sqrt{d_k}$ 로 나누면 점수의 표준편차가 지나치게 커지는
것을 막는다. 그렇지 않으면 소프트맥스 출력이 0 또는 1에 매우 가까워지고 기울기가 작아져
학습이 불안정해질 수 있다.

5.3 마스크를 더하기

M 은 허용 위치에 0, 금지 위치에 매우 작은 값(이론적으로 $-\infty$)을 둔다. 소프트맥스를
거치면 금지 위치의 확률은 0이 된다.

5.4 행별 소프트맥스

점수 행 s_i 의 j 번째 가중치는 다음과 같다.

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{r=1}^T \exp(s_{ir})} \quad (5)$$

따라서 $a_{ij} \geq 0$ 이고 $\sum_j a_{ij} = 1$ 이다. 각 출력은 값 벡터의 볼록 가중합이 된다.

$$\mathbf{o}_i = \sum_{j=1}^T a_{ij} \mathbf{v}_j \quad (6)$$

6. Notebook과 같은 3개 토큰 수치 예제

[week04.ipynb](#)의 첫 실습은 계산을 투명하게 보기 위해

$Q = K = V = X$ 로 둔다.

$$X = \begin{bmatrix} 1 & 0 \\ 0.8 & 0.2 \\ 0 & 1 \end{bmatrix}, \quad d_k = 2 \quad (7)$$

점수 행렬을 계산하면 다음과 같다.

$$S = \frac{XX^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.707 & 0.566 & 0.000 \\ 0.566 & 0.481 & 0.141 \\ 0.000 & 0.141 & 0.707 \end{bmatrix} \quad (8)$$

각 행에 소프트맥스를 적용한 어텐션 가중치는 약

$$A \approx \begin{bmatrix} 0.424 & 0.368 & 0.209 \\ 0.389 & 0.357 & 0.254 \\ 0.239 & 0.276 & 0.485 \end{bmatrix} \quad (9)$$

이다. 모든 행의 합은 반올림 오차를 제외하면 1이다. AX 를 계산하면

$$H \approx \begin{bmatrix} 0.718 & 0.282 \\ 0.674 & 0.326 \\ 0.460 & 0.540 \end{bmatrix} \quad (10)$$

가 된다. 첫 출력도 첫 입력 $[1, 0]$ 의 복사가 아니라 세 값의 가중합이다. 이것이 “문맥을 섞는다”는 말의 수학적 의미다. 실제 모델에서는 학습된 투영과 여러 층의 비선형 변환이 더해진다.

Notebook 점검 1

1. `weights.sum(axis=1)` 이 1인지 확인한다.
2. 둘째 입력의 두 좌표를 바꾼 뒤, 한 위치의 변화가 세 출력에 모두 나타나는지 관찰한다.
3. 실행 결과를 위 손계산과 비교하고, 차이가 있다면 반올림인지 구현 오류인지 구분한다.

7. 두 종류의 마스크

패딩 마스크(padding mask)는 입력 길이를 맞추기 위해 넣은 채움 토큰을 계산에서 제외한다. Notebook의 `key_mask=[1,1,0]` 은 셋째 키 열을 가린다. 그 결과 모든 질의 행에서 셋째 열의 가중치는 정확히 0이 된다.

인과 마스크(causal mask)는 디코더가 다음 토큰을 학습할 때 미래 위치를 보지 못하게 한다. 위치 i 는 $j \leq i$ 만 볼 수 있다. 인과 마스크가 없다면 학습 시 정답 미래 토큰이 입력으로 노출되어, 배포 시에는 재현할 수 없는 데이터 누수가 생긴다.

두 마스크는 함께 사용될 수도 있다. 하나는 “실제 입력이 아닌 위치”를, 다른 하나는 “시간상 아직 볼 수 없는 위치”를 막는다.

8. 다중 헤드 어텐션

하나의 어텐션만 쓰지 않고 H 개의 투영 공간에서 병렬로 계산할 수 있다.

$$\text{head}_h = \text{Attention}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)}) \tag{11}$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O \tag{12}$$

각 헤드는 다른 관계를 포착할 **용량**을 얻지만, “3번 헤드는 문법만 담당한다”처럼 고정된 인간 개념으로 해석할 수는 없다. 헤드 수를 늘리면 헤드당 차원과 메모리 접근 패턴도 달라지므로 정확도만이 아니라 처리량과 지연시간을 함께 측정해야 한다.

9. 순서 정보는 어디에서 오는가

어텐션 자체는 입력 행의 순서를 바꾸면 출력도 같은 방식으로 바뀌는 순열 등변 연산이다.
“학생이 교수를 도왔다”와 “교수가 학생을 도왔다”를 구분하려면 위치 정보가 필요하다.

원 Transformer의 사인·코사인 위치 인코딩은 다음과 같다.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d}), \quad PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d}) \quad (13)$$

서로 다른 주기의 파동을 조합해 각 위치를 표현한다. 이후 모델에서는 학습형 절대 위치, 상대 위치 편향, RoPE(rotary positional embedding), ALiBi 등 다양한 방법을 사용한다. 최대 입력 길이가 길다는 사양만으로 장문 이해가 좋다고 결론 내릴 수 없다. 가운데 정보 손실, 위치 외삽, 첫 토큰 처리 비용, 메모리 사용량을 실제 장문 과제로 평가해야 한다.

10. Transformer 블록은 어텐션만이 아니다

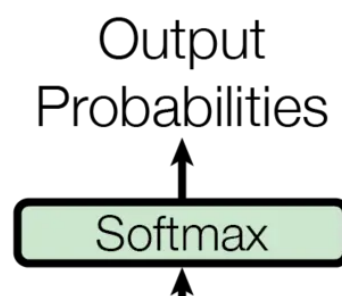
전형적인 블록에는 다음 구성요소가 있다.

1. 다중 헤드 자기 어텐션
2. 잔차 연결(residual connection)
3. 층 정규화(layer normalization)
4. 위치별 피드포워드 네트워크(feed-forward network, FFN)

잔차 연결은 $X + F(X)$ 처럼 입력을 우회 경로로 더한다. 깊은 네트워크에서 원래 정보와 기울기가 흐를 통로를 만든다. 층 정규화는 한 토큰 벡터의 특징 차원에 대해 평균과 분산을 조절한다. 배치 전체를 기준으로 하는 배치 정규화와 축이 다르다.

FFN은 각 위치에 같은 가중치를 적용하지만 위치 사이를 직접 섞지는 않는다. 어텐션이 토큰 사이의 정보를 모으면, FFN은 각 위치 내부의 특징을 비선형적으로 변환한다. 실제 모델은 사전 층 정규화와 사후 층 정규화, GELU/SwiGLU 같은 활성화·게이트 방식에서 차이가 있다.

11. 논문 도판 읽기: Transformer 전체 구조



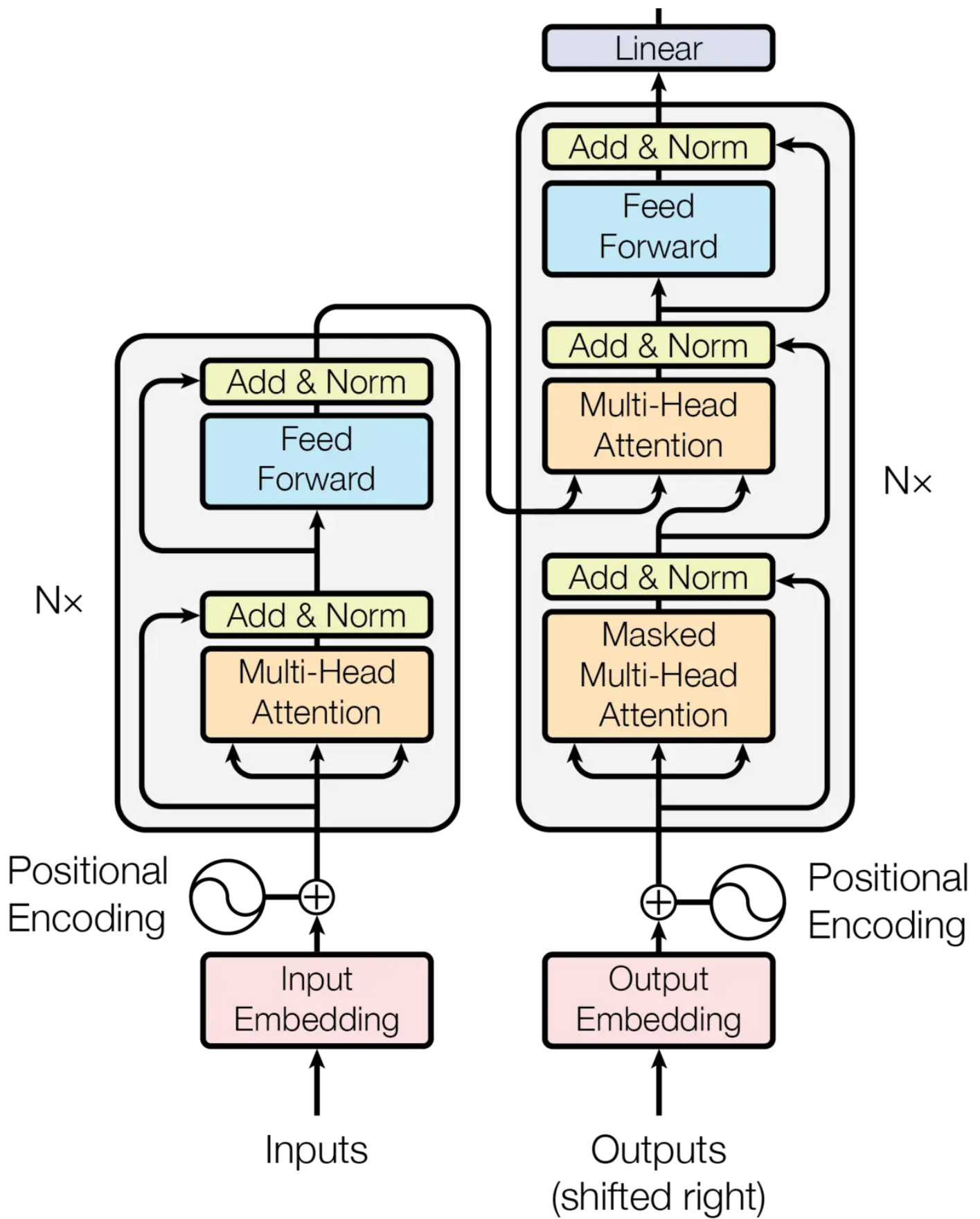


그림 1. Vaswani et al. (2017), "Attention Is All You Need," Figure 1. 왼쪽은 인코더, 오른쪽은 자기회귀 디코더다. [원문과 Figure 1](#)

그림은 아래에서 위로 읽는다.

- 왼쪽 아래 입력 임베딩과 위치 표현의 합이 인코더로 들어간다.
- 인코더는 가리지 않은 자기 어텐션과 FFN을 반복해 원문의 문맥 표현을 만든다.
- 오른쪽 아래 디코더는 한 칸 이동한 이전 출력 토큰을 입력으로 받는다.
- 디코더의 첫 어텐션은 미래를 가린 다중 헤드 자기 어텐션이다.
- 그 위의 인코더-디코더 어텐션은 디코더 상태를 질의로, 인코더 출력을 키와 값으로 사용한다.
- 마지막 선형층과 소프트맥스가 다음 토큰의 확률 분포를 만든다.

그림의 반복 표시 $N \times$ 는 블록을 여러 층 쌓는다는 뜻이다. 화살표가 바깥으로 우회해 Add & Norm 에 들어가는 부분은 잔차 연결이다. 이 도판은 “Transformer=어텐션 하나”가 아니라 여러 연산이 연결된 구조라는 점을 보여준다.

12. 인코더 전용, 디코더 전용, 인코더-디코더

구조	대표 모델	문맥 범위	주요 용도
인코더 전용	BERT 계열	보통 양방향	분류, 개체명 인식, 검색 표현
디코더 전용	GPT, Phi 계열	왼쪽 문맥	다음 토큰 생성
인코더-디코더	T5, 원 Transformer	인코더 양방향 + 디코더 인과적	번역, 조건부 생성

BERT의 가려진 언어 모델링(masked language modeling, MLM)은 일부 입력 토큰을 가린 뒤 좌우 문맥으로 복원하게 한다. `last_hidden_state` 는 토큰별 표현이므로 개체명 인식 등에 직접 쓸 수 있다. `[CLS]` 나 평균 풀링으로 문장 벡터를 만들 수는 있지만, 사전학습 BERT가 의미 유사도에 맞는 기하를 자동으로 만든다는 보장은 없다. 이 문제를 5주차의 SBERT가 다룬다.

13. 생성 과정: 은닉 상태에서 다음 토큰까지

Hands-On Large Language Models 3장의 생성 코드는 다음 데이터 흐름으로 읽는다.

문자열 → 토큰 ID → 은닉 상태 → 로짓(logit) → 확률 분포 → 토큰 선택 → 입력 뒤에 붙여 반복

온도(temperature) τ 는 로짓 l_i 를 l_i/τ 로 바꾼 뒤 소프트맥스를 적용한다.

$\tau < 1$ 이면 분포가 뾰족해지고, $\tau > 1$ 이면 평평해진다. 온도는 토큰 임베딩 자체를 바꾸는 설정이 아니라 **선택 단계의 분포**를 조정한다.

누적 확률 표본 추출(top-p sampling)은 확률이 큰 토큰부터 합해 누적 질량 p 를 넘기는 최소 후보 집합에서 표본을 뽑는다. 같은 은닉 상태라도 디코딩 규칙에 따라 출력 문장은 달라질 수 있다.

14. KV 캐시와 검색 색인은 다르다

자기회귀 생성의 t 번째 단계에서 과거 토큰의 키와 값은 이미 계산되어 있다. KV 캐시는 이를 재사용해 새 토큰의 질의·키·값만 추가한다. 매 단계마다 전체 접두문을 다시 계산하지 않아 생성이 빨라지지만, 층 수·헤드 수·문맥 길이·자료형에 따라 캐시 메모리는 증가한다.

검색용 벡터 색인은 문서 임베딩을 장기간 저장하고 여러 질의에서 재사용하는 자료구조다. 반면 KV 캐시는 한 생성 요청의 중간 상태이며 모델 층별 키와 값을 저장한다. 목적, 수명, 벡터의 의미가 모두 다르다.

15. 생성 모델의 은닉 상태를 문장 벡터로 쓸 때

토큰 은닉 상태 $H \in \mathbb{R}^{T \times d}$ 를 평균하면 한 벡터가 되지만, 다음을 확인해야 한다.

- 채움 토큰을 마스크로 제외했는가?
- EOS·BOS 같은 특수 토큰을 포함할 것인가?
- 인과 모델의 앞 위치와 뒤 위치가 본 문맥 범위가 다름을 고려했는가?
- 학습목표가 의미 유사도나 검색 관련성을 직접 최적화했는가?
- 출력 벡터를 정규화했고, 모델 카드의 권장 접두문을 사용했는가?

[week04.ipynb](#)의 `plain_mean` 과 `masked_mean` 은 채움 위치에

`[9, 9]`를 넣어 이 문제를 의도적으로 크게 만든다. 단순 평균은 `[3.6, 3.067]`이지만 마스크 평균은 `[0.9, 0.1]`이다. 패딩 값이 바뀌어도 유지되어야 하는 것은 마스크 평균뿐이다.

Notebook 점검 2

1. 채움 벡터를 `[100, -100]`으로 바꿔 두 평균을 다시 비교한다.
2. `RUN_TRANSFORMER=True` 선택 실습에서는 `[batch, tokens, hidden]` 모양을 기록한다.

3. 동형이의어가 있는 한국어 문장 두 쌍을 만들고, 같은 토큰 위치의 코사인 유사도를 비교한다.
4. 모델명, 토큰화 결과, 풀링 방식, 벡터 차원을 실험 기록에 남긴다.

16. 최신 연구를 읽는 관점

LLM2Vec은 디코더 전용 LLM에 양방향 어텐션, 가려진 다음 토큰 예측, 대조학습을 적용해 범용 텍스트 표현기로 전환하는 방법을 제안했다. GritLM은 생성과 표현 학습을 한 모델에 결합하는 방향을 탐구했다. 공통 질문은 “생성 능력이 좋은 은닉 상태를 어떻게 비교 가능한 벡터 공간으로 정렬할 것인가?”이다.

이런 모델을 평가할 때는 매개변수 수나 평균 순위만 보지 않는다. 한국어, 장문, 비대칭 검색, 분류, 군집화 등 실제 과업과 처리량·메모리·지연시간을 함께 고정해야 한다.

17. 어텐션 가중치는 설명인가

어텐션 행렬은 특정 층과 헤드에서 어떤 값이 얼마나 섞였는지를 보여준다. 그러나 큰 가중치가 최종 예측에 큰 **인과적 영향**을 주었다고 곧바로 결론 내릴 수 없다. 이후 층, 잔차 경로, 값 벡터, FFN이 결과를 바꿀 수 있기 때문이다.

실습에서는 “부정어에 높은 가중치가 있다”는 관찰과 “부정어 때문에 예측이 바뀌었다”는 주장을 구분한다. 후자를 주장하려면 토큰 제거·대체, 기울기 기반 분석, 개입 실험 등 추가 근거가 필요하다.

18. 자주 하는 오해

1. **입력 임베딩이 곧 문맥 임베딩이다.** 입력 조희 벡터는 문맥 전이며, 은닉 상태가 문맥화된 표현이다.
2. **어텐션 가중치는 단어 유사도다.** 학습된 질의-키 호환성이지만 일반적인 의미 유사도 점수가 아니다.
3. **각 헤드는 사람이 이름 붙일 수 있는 기능 하나를 담당한다.** 일부 패턴은 관찰되지만 고정 기능은 보장되지 않는다.
4. **긴 최대 문맥 길이는 장문 이해를 보장한다.** 위치 활용과 검색 능력은 실제 과제로 따로 평가해야 한다.
5. **평균 풀링만 하면 모든 LLM이 좋은 검색 모델이 된다.** 학습목표와 풀링·지시문이 맞아야 한다.
6. **KV 캐시는 벡터 데이터베이스다.** 생성 중간 상태와 문서 검색 색인은 서로 다른 자료구조다.
7. **어텐션 그림이 모델의 사고 과정을 증명한다.** 정보 흐름의 단서일 뿐 충분한 인과 설명이 아니다.

19. 핵심 정리

- 문맥 임베딩은 토큰 열 전체를 조건으로 위치마다 계산되는 은닉 벡터다.
- 자기 어텐션은 질의-키 내적으로 가중치를 만들고 값 벡터를 가중합한다.
- $1/\sqrt{d_k}$ 는 점수 규모와 소프트맥스 포화를 조절한다.
- 패딩 마스크와 인과 마스크는 각각 가짜 위치와 미래 정보의 유입을 막는다.
- Transformer의 표현력은 어텐션, 위치 정보, 잔차 연결, 층 정규화, FFN의 결합에서 나온다.
- 인코더와 디코더는 볼 수 있는 문맥과 학습목표가 다르다.
- 생성 은닉 상태를 검색 벡터로 사용할 때는 목적 특화 학습과 평가가 필요하다.

20. 복습 문제

1. Q, K, V 가 모두 X 에서 만들어지는데도 서로 다른 역할을 할 수 있는 이유를 쓰시오.
2. $Q \in \mathbb{R}^{5 \times 8}, K \in \mathbb{R}^{5 \times 8}$ 일 때 $QK^\top$ 의 모양은 무엇인가?
3. d_k 가 커질수록 스케일 조정이 필요한 이유를 분산 관점에서 설명하시오.
4. 패딩 마스크와 인과 마스크의 금지 위치를 각각 예로 그리시오.
5. Figure 1에서 인코더-디코더 어텐션의 질의, 키, 값은 어디에서 오는가?
6. BERT의 양방향 은닉 상태와 GPT의 인과적 은닉 상태가 문장 풀링에 주는 차이를 설명하시오.
7. 온도와 top-p가 임베딩이 아니라 디코딩 규칙이라는 말을 설명하시오.
8. KV 캐시와 검색용 임베딩 색인을 수명·목적·저장 내용으로 비교하시오.
9. 어텐션 가중치를 모델 설명으로 사용할 때 필요한 추가 검증 두 가지를 제시하시오.

참고문헌

1. Vaswani, A. et al. (2017). "Attention Is All You Need." [arXiv](#)
2. Devlin, J. et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." [ACL Anthology](#)
3. Su, J. et al. (2021). "RoFormer: Enhanced Transformer with Rotary Position Embedding." [arXiv](#)

4. Dao, T. et al. (2022). “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” [NeurIPS](#)
5. Jain, S. & Wallace, B. C. (2019). “Attention is not Explanation.” [NAACL](#)
6. BehnamGhader, P. et al. (2024). “LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders.” [arXiv](#)
7. Muennighoff, N. et al. (2024). “Generative Representational Instruction Tuning.” [arXiv](#)
8. Hands-On LLM Authors. *Hands-On Large Language Models*, [Chapter 2](#) and [Chapter 3](#)